

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I VENNA MEENAKSHI REDDY(192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Aim

To process an SQL query from user input until the final result is produced.

Pseudocode

ALGORITHM Query_Processing(SQL_Query)

INPUT:

SQL_Query

OUTPUT:

Query_Result or Error_Message

BEGIN

STEP 1: Receive SQL Query

Read SQL_Query

STEP 2: Lexical and Syntax Analysis

Divide SQL_Query into tokens

Check SQL syntax

IF syntax is invalid THEN

Display "Syntax Error"

STOP

END IF

STEP 3: Semantic Analysis

Identify tables and columns

Check whether tables exist

Check whether columns exist

Check data types and permissions

IF semantic error exists THEN

Display "Semantic Error"

STOP

END IF

STEP 4: Query Translation

Convert SQL query into an internal representation
such as relational algebra

STEP 5: Query Optimization

- Generate possible execution plans
- Estimate the cost of each plan
- Select the plan with minimum cost

STEP 6: Query Execution

- Send the selected execution plan to the execution engine
- Access required data from the database
- Perform selections, projections, joins, sorting, etc.

STEP 7: Result Generation

- Collect the output tuples

STEP 8: Display Result

- Display Query_Result to the user

END

Flow

SQL Input



Parsing



Semantic Analysis



Relational Algebra



Query Optimization



Best Execution Plan



Query Execution



Result Output

2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Aim

To compare different join orders based on estimated I/O cost and select the cheapest execution plan.

Pseudocode

ALGORITHM Cost_Based_Optimization(Relations)

INPUT:

Relations R1, R2, ..., Rn

Number of buffer pages B

Number of pages in each relation

OUTPUT:

Best_Join_Order

Minimum_Cost

BEGIN

Generate all possible join orderings

Minimum_Cost $\leftarrow$ INFINITY

Best_Join_Order $\leftarrow$ NULL

FOR each Join_Order in Possible_Join_Orders DO

 Total_Cost $\leftarrow$ 0

 Current_Result_Pages $\leftarrow$ pages of first relation

 FOR each next relation R in Join_Order DO

 Cost $\leftarrow$ Estimate_Join_Cost(Current_Result_Pages,
 pages(R),
 B)

 Total_Cost $\leftarrow$ Total_Cost + Cost

 Estimate size of join result

 Current_Result_Pages $\leftarrow$ Estimated_Result_Pages

END FOR

IF Total_Cost < Minimum_Cost THEN
 Minimum_Cost $\leftarrow$ Total_Cost
 Best_Join_Order $\leftarrow$ Join_Order
END IF

END FOR

RETURN Best_Join_Order, Minimum_Cost

END

I/O Cost Formula

For a simple block nested-loop join:

$$\text{Cost} = B(R) + \text{ceil}(B(R) / (M - 2)) \times B(S)$$

Where:

- $B(R)$ = number of pages in relation R
- $B(S)$ = number of pages in relation S
- M = number of available buffer pages

Example

Suppose:

$R = 100$ pages

$S = 50$ pages

$M = 10$ buffers

Then:

$$\begin{aligned}\text{Cost} &= 100 + \text{ceil}(100 / 8) \times 50 \\ &= 100 + 13 \times 50 \\ &= 750 \text{ I/O}\end{aligned}$$

The optimizer compares such costs for different join orders and chooses the minimum-cost plan.

3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Aim

To join two relations by comparing each tuple of the outer relation with every tuple of the inner relation.

Pseudocode

ALGORITHM Nested_Loop_Join(R, S, Join_Condition)

INPUT:

Relation R
Relation S
Join_Condition

OUTPUT:

Joined_Relation

BEGIN

Result $\leftarrow$ Empty

FOR each tuple r in R DO

FOR each tuple s in S DO

IF Join_Condition(r, s) = TRUE THEN
Add combined tuple (r, s) to Result
END IF

END FOR

END FOR

RETURN Result

END

Example

R:

1 Meena

2 Ravi

3 Anu

S:

1 90

2 85

4 70

Join condition:

$R.id = S.id$

Result:

1 Meena 90

2 Ravi 85

Time Complexity

If:

R contains M tuples

S contains N tuples

Then:

Time Complexity = $O(M \times N)$

For page-based block nested-loop join:

I/O Cost =

$B(R) + \text{ceil}(B(R)/(M-2)) \times B(S)$

where M represents the number of buffer pages.

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Aim

To detect whether transactions are involved in a deadlock by checking for a cycle in the Wait-For Graph.

Concept

If:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T1$

then a cycle exists.

Therefore:

Cycle = Deadlock

Pseudocode

ALGORITHM Detect_Deadlock(WFG)

INPUT:

Wait-For Graph WFG

OUTPUT:

Deadlock detected or No deadlock

BEGIN

FOR each transaction T DO

visited[T] $\leftarrow$ FALSE

recursion_stack[T] $\leftarrow$ FALSE

END FOR

FOR each transaction T DO

IF visited[T] = FALSE THEN

IF DFS_Cycle_Detection(T) = TRUE THEN

Display "Deadlock Detected"

RETURN TRUE

END IF

END IF

END FOR

Display "No Deadlock"

RETURN FALSE

END

DFS Algorithm

ALGORITHM DFS_Cycle_Detection(T)

BEGIN

visited[T] ← TRUE

recursion_stack[T] ← TRUE

FOR each transaction U connected from T DO

IF visited[U] = FALSE THEN

IF DFS_Cycle_Detection(U) = TRUE THEN

RETURN TRUE

END IF

ELSE IF recursion_stack[U] = TRUE THEN

RETURN TRUE

END IF

END FOR

recursion_stack[T] ← FALSE

RETURN FALSE

END

Example

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T1$

DFS finds:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Therefore:

Deadlock Detected

5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Aim

To maintain serializability by dividing transaction execution into a Growing Phase and a Shrinking Phase.

Pseudocode

ALGORITHM Two_Phase_Locking(Transaction T)

INPUT:

Transaction T

Data items required by T

OUTPUT:

Committed or Aborted Transaction

BEGIN

Phase $\leftarrow$ GROWING

FOR each operation of Transaction T DO

IF operation requires a lock THEN

IF Phase = GROWING THEN

Request appropriate lock

IF lock is available THEN

Grant lock

ELSE

Transaction waits

END IF

ELSE

Display "Cannot acquire new lock"

ABORT Transaction

RETURN

END IF

END IF

Execute operation

IF Transaction decides to release a lock THEN

Phase $\leftarrow$ SHRINKING

Release the lock

END IF

END FOR

Commit Transaction

Release all remaining locks

END

Important rule

Growing Phase:

Locks can be acquired

Locks are not released

Shrinking Phase:

Locks can be released

New locks cannot be acquired

Example

T1:

Lock(A)

Lock(B)

Read(A)

Write(B)

Unlock(A)

Unlock(B)

Commit

Once Unlock(A) occurs, T1 cannot acquire another lock.

6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Aim

To handle conflicting write operations using transaction timestamps.

For each data item X:

Read_TS(X)

Write_TS(X)

are maintained.

Pseudocode

ALGORITHM Thomas_Write_Rule(T, X)

INPUT:

Transaction T

Data item X

Timestamp TS(T)

Read_TS(X)

Write_TS(X)

OUTPUT:

Write Accepted, Write Ignored, or Transaction Aborted

BEGIN

IF $TS(T) < Read_TS(X)$ THEN

Display "Transaction is too old"

ABORT Transaction

RETURN

ELSE IF $TS(T) < Write_TS(X)$ THEN

Display "Obsolete write"

Ignore the write

RETURN

ELSE

Perform WRITE(X)

Write_TS(X) $\leftarrow$ TS(T)

Display "Write Accepted"

RETURN

END IF

END

Example

Suppose:

TS(T1) = 10

Read_TS(X) = 5

Write_TS(X) = 15

Since:

TS(T1) < Write_TS(X)

10 < 15

the write is obsolete.

Thomas' Write Rule says:

Ignore the write

instead of unnecessarily aborting the transaction.

7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

ARIES stands for:

Algorithm for Recovery and Isolation Exploiting Semantics

It has three major phases:

1. Analysis
2. Redo
3. Undo

Pseudocode

ALGORITHM ARIES_Recovery(Log)

INPUT:

Database Log

OUTPUT:

Recovered Database

BEGIN

PHASE 1: ANALYSIS

Read log from the most recent checkpoint

Initialize Transaction Table

Initialize Dirty Page Table

FOR each log record DO

 Update Transaction Table

 IF record modifies a page THEN

 Add page to Dirty Page Table if necessary

 END IF

 IF transaction commits THEN

 Mark transaction as committed

END IF

END FOR

Identify:

Winner Transactions

Loser Transactions

PHASE 2: REDO

Find the starting point for REDO

FOR each log record from REDO starting point DO

IF operation needs to be redone THEN

Repeat the operation

END IF

END FOR

PHASE 3: UNDO

FOR each loser transaction DO

Process its log records in reverse order

Undo every uncommitted operation

Write Compensation Log Record

END FOR

Write recovery completion information

RETURN recovered database

END

Example

Log:

T1 WRITE A

T2 WRITE B

T1 COMMIT

T2 WRITE C

CRASH

After crash:

T1 = committed → REDO

T2 = uncommitted → UNDO

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Aim

To recover the database without traditional undo/redo logging by maintaining a shadow page table and a current page table.

Pseudocode

ALGORITHM Shadow_Paging_Recovery

INPUT:

Database Pages

OUTPUT:

Committed Database State

BEGIN

STEP 1:

Create Shadow_Page_Table

STEP 2:

Copy Shadow_Page_Table into Current_Page_Table

STEP 3:

FOR every update operation DO

Create a new page for modified data

Update Current_Page_Table

Do NOT modify the shadow page

END FOR

STEP 4:

IF transaction COMMIT occurs THEN

Make Current_Page_Table the new
Shadow_Page_Table

Display "Commit Successful"

ELSE IF system FAILURE occurs THEN

Discard Current_Page_Table

Restore Shadow_Page_Table

Display "Rollback Successful"

END IF

END

Example

Initial:

Shadow Page Table

Page 1 → A

Page 2 → B

Page 3 → C

After update:

Current Page Table

Page 1 → A

Page 2 → B_New

Page 3 → C

If failure occurs:

Current Page Table → Discarded

Shadow Page Table → Restored

9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Aim

To recover the database after a crash using the log records.

Pseudocode

ALGORITHM Immediate_Update_Recovery(Log)

INPUT:

Recovery Log

OUTPUT:

Recovered Database

BEGIN

Read all log records

Create REDO_List

Create UNDO_List

FOR each transaction in Log DO

IF transaction has COMMIT record THEN

Add transaction to REDO_List

ELSE

Add transaction to UNDO_List

END IF

END FOR

REDO PHASE

FOR each transaction in REDO_List DO

FOR each WRITE operation of transaction
in forward order DO

REDO the operation

END FOR

END FOR

UNDO PHASE

FOR each transaction in UNDO_List DO

FOR each WRITE operation of transaction
in reverse order DO

Restore the old value

END FOR

END FOR

Display "Recovery Completed"

END

Example

Log:

<T1, A, Old=100, New=200>

<T2, B, Old=300, New=400>

<T1, COMMIT>

CRASH

Analysis:

T1 → committed → REDO

T2 → uncommitted → UNDO

Therefore:

REDO T1

UNDO T2

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Aim

To design a database connectivity manager that handles:

- Connection pooling
- Connection acquisition
- Query execution
- Commit
- Exception handling
- Rollback
- Connection release

Pseudocode

ALGORITHM Database_Connectivity_Manager

INPUT:

Database details

SQL Query

OUTPUT:

Query Result or Error

BEGIN

STEP 1: CREATE CONNECTION POOL

Create a pool containing multiple database connections

STEP 2: GET CONNECTION

Request a connection from the pool

IF no connection is available THEN

 Wait until a connection becomes available

END IF

STEP 3: EXECUTE QUERY

TRY

Create database cursor

Execute SQL Query

IF query modifies data THEN

Commit transaction

END IF

Store query result

STEP 4: EXCEPTION HANDLING

CATCH Database_Exception

Rollback transaction

Display error message

STEP 5: RELEASE CONNECTION

Return connection to the connection pool

RETURN query result

END

Flow

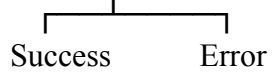
Create Connection Pool



Get Connection



Execute Query



Success

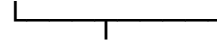
Error



Commit



Rollback



Release Connection



Result